

Open/Closed- & Liskov Substitution-Principle

SWEN1 | BIF3

Table of Contents

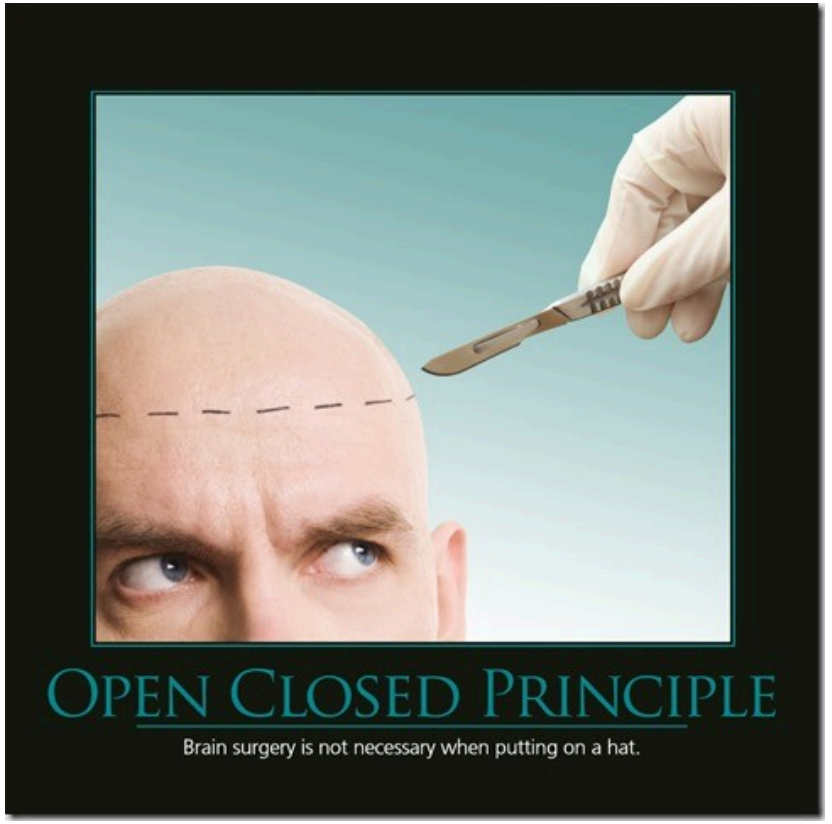
- Open/Closed Principle
- Liskov‘s Substitution Principle



Open/Closed Principle

Open/Closed Principle

- Bertrand Meyer is generally credited for having originated the term, which appeared in his 1988 book Object Oriented Software Construction. (https://en.wikipedia.org/wiki/Open-closed_principle)
- *"software entities … should be **open for extension**, but **closed for modification**"*



Language Support

The Open/Closed-Principle is used in the concept of inheritance of object-oriented programming

- With **inheritance** the functionality of a class is extended.
  The base class stays the same, the derived class may
  - Add functionality by implementing **new methods**
  - Modify functionality by **overwriting existing** methods
- Language Support with `sealed` **classes and interfaces**
- **Pattern matching** (records)

Extensibility ≠ Modification

- Inheritance **adds** behaviour; it should not **switch** on types
- Prefer composition or strategy pattern for variability
- Inheritance is justified when *is-a & behaves-as* both hold

Sample: PaymentService (1/2)

```csharp
// C# Sample (excerpt):
public enum PaymentType { Card, BankTransfer, Crypto }

public class PaymentService
{
    public Receipt Pay(PaymentRequest req)
    {
        switch (req.Type)
        {
            case PaymentType.Card:
                return new CardGateway().Pay(req);
            case PaymentType.BankTransfer:
                return new SepaGateway().Pay(req);
            case PaymentType.Crypto:
                return new CryptoGateway().Pay(req);
            default:
                throw new NotSupportedException("New type?");
        }
    }
}
```

```java
// Java Sample (excerpt):
public enum PaymentType { CARD, BANK_TRANSFER, CRYPTO }

public class PaymentService
{
    public Receipt pay(PaymentRequest req)
    {
        switch (req.Type)
        {
            case PaymentType.CARD:
                return new CardGateway().pay(req);
            case PaymentType.BANK_TRANSFER:
                return new SepaGateway().pay(req);
            case PaymentType.CRYPTO:
                return new CryptoGateway().pay(req);
            default:
                throw new NotSupportedException("New type?");
        }
    }
}
```

OCR violated!

Payment-Types are added through re-writing the Pay implementation (adding new switch cases.)

Sample: PaymentService (2/2)

```csharp
// C# Sample (excerpt)
public interface IPaymentProcessor
{
    Receipt Pay(PaymentRequest req);
    PaymentType SupportedType { get; }
}

public class PaymentService
{
    private readonly IDictionary<PaymentType, IPaymentProcessor> _map;
    public PaymentService(IEnumerable<IPaymentProcessor> processors)
        => _map = processors.ToDictionary(p => p.SupportedType);

    public Receipt Pay(PaymentRequest req) =>
        _map[req.Type].Pay(req); // unchanged when adding new processors
}
```

```java
// Java Sample (excerpt)
public interface PaymentProcessorInterface {
    Receipt pay(PaymentRequest req);
    PaymentType getSupportedType();
}

public final class PaymentService {
    private final Map<PaymentType, PaymentProcessorInterface> map;

    public PaymentService(Collection<PaymentProcessorInterface> processors) {
        this.map = processors.stream()
            .collect(Collectors.toUnmodifiableMap(
                PaymentProcessor::getSupportedType,
                Function.identity()
            ));
    }

    public Receipt pay(PaymentRequest req) {
        PaymentProcessor processor = map.get(req.getType());
        return processor.pay(req);
    }
}
```

New processors implement `IPaymentProcessor`; no edits to `PaymentService`.

Sample: Sealed Shape Hierarchy

```csharp
// C# Sample
public abstract class Shape {
    double Area();
}

public class Circle(double r) : Shape {
    public double Area() { return Math.PI * r * r; }
}

public class Square(double side) : Shape {
    public double Area() { return side * side; }
}
```

```java
// Java Sample
public sealed interface Shape permits Circle, Square {
    double area();
}

public record Circle(double r) implements Shape {
    public double area() { return Math.PI * r * r; }
}

public record Square(double side) implements Shape {
    public double area() { return side * side; }
}
```

Hierarchy is closed to unforeseen extensions yet open to new behaviour via default methods, visitors, etc.

Anti-pattern Spotlight

- ❌ Exhaustive `if (x instanceof …)` chains
- ❌ Enum-driven logic switches
- ❌ Editing legacy classes on every new requirement
- ❌ Global variables
  - When a global variable is used in a class or method, then the class is not closed against modifications of the structure of the global variable.
  - All classes and methods should be closed, so global variables are not necessary!

Smell: “Closed for extension” design

Testing Pay-off

- add `ApplePayProcessor` before refactor ⇒ 7 unit-tests changed
- add `ApplePayProcessor` after OCP refactor ⇒ 0 unit-tests changed

Stable tests signal stable design.

OCP - Key Take-aways

“Software entities (modules, classes) should be open for extension, but closed for modification.”

1. Add behaviour with new types, not new branches
2. Guard invariants with strong interfaces
3. Favour dependency injection for discoverability
4. Review classes: Would adding a variant require editing this file?

Challenges when using OCP:

- Encapsulation – only make methods/attribute public when necessary!
- Interface vs Inheritance

Further Reading

- dev.to: SOLID: Open-Closed Principle (OCP) in C#



Liskov‘s Substitution Principle

Liskov‘s Substitution Principle

- initially introduced by Barbara Liskov & Jeannette Wing in a 1987 conference keynote address titled Data abstraction and hierarchy. (https://en.wikipedia.org/wiki/Liskov_substitution_principle)
- *“Subtypes must be substitutable for their base types without altering the correctness of the program.”*
  or
- *“objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program”*
- Liskov‘s substitution principle meets Open/Closed principle
  A violation of LSP may easily lead to a violation of OCP!

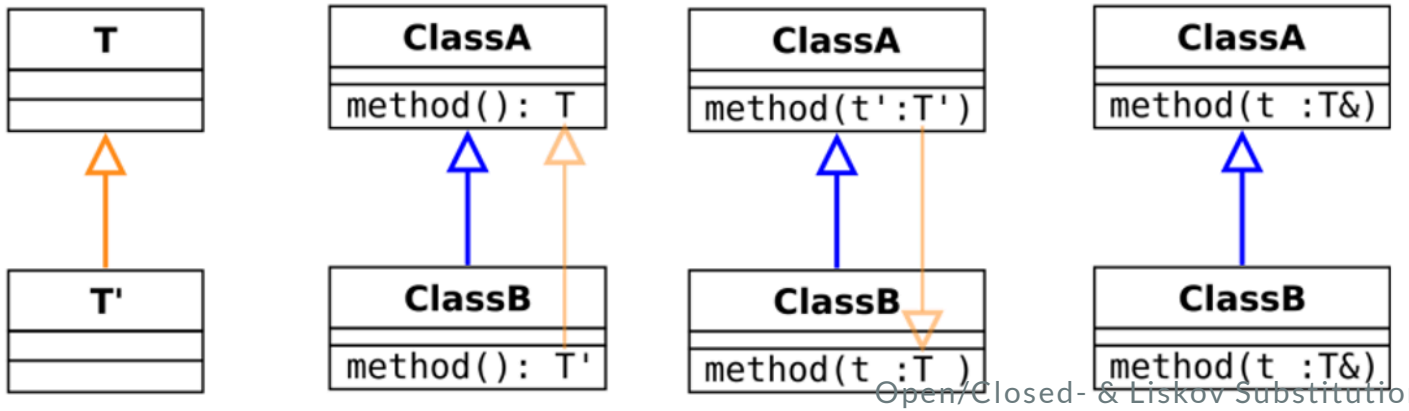


Behaviour > Signatures

- **Design by Contract (DbC)**: base preconditions ≤ subtype preconditions
- Postconditions & invariants must be preserved or strengthened
- Violations often surface only in behavioural tests

Covariance and Contravariance

- **Covariance**:
  Type of element in sub-type is sub-type of the element-type in the base-type.
  Enables you to pass a derived type where a base type is expected. --> applied to **return-types**
- **Contravariance**:
  Type of element in sub-type is base-type of the element-type in the base-type.
  Allows a method with the parameter of a base class to be assigned to a delegate that expects the parameter of a derived class. --> applied to **parameters**
- **Invariance**:
  Type of element in sub-type is equivalent to the element-type in the base-type.
  The type to be assigned and the type assigned must be the same.



Covariance & Contravariance (C#)

```
class Animal {...}
class Dog : Animal {...}

IEnumerable<Dog> dogs = new List<Dog>();
IEnumerable<Animal> animals = dogs; // ✓ covariance (out)

Action<Animal> feed = a => a.Eat();
Action<Dog>  feedDog = feed; // ✗ contravariance (in) mismatch
```

Variance helps or hinders substitutability—understand the arrows.

PECS & Generics (Java)

```
void copy(List<? extends Fruit> src, List<? super Fruit> dest) { … }
```

- **Producer `extends`, Consumer `super`** (=PECS)
- Ensures method honours LSP despite generic wildcards

Immutable vs Mutable Lists (Java)

Passing `List` where `ImmutableList` expected lets clients mutate and violate invariants → subtle LSP break.
✓ Provide *read-only view* (`Collections.unmodifiableList`) or use sealed hierarchy.

Classic Violation – Rectangle ⇒ Square

```
class Rectangle {...}
class Square extends Rectangle {...}

Rectangle r = new Square(5);
r.setWidth(4);
assert r.area() == 20; // ✅ for rectangle, ❌ for square
```

Square strengthens invariants (equal sides) ⇒ breaks callers' expectations.

Refactor – Composition over Inheritance

```java
interface Shape { double area(); }

final class Square implements Shape {
    private final double side;
    Square(double side) { this.side = side; }
    public double area() { return side * side; }
}

final class ResizableRectangle implements Shape {
    private double w, h;
    // setters maintain own invariants without affecting Square
}
```

Separate hierarchies; behaviour preserved.

NewsletterMailer (C#) – Exception Contracts

```
class BaseMailer { virtual void Send() throws MailException; }
class HtmlMailer : BaseMailer { override void Send() throws IOException; } // ❌
```

Subtype widens thrown exceptions ⇒ callers must now handle more cases.

LSP and Unit-Testing

- Every test written for the base type must pass for every subtype
- Additional subtype tests are allowed, but no edits to base tests
- Mutation testing surfaces hidden LSP issues

Design-By-Contract

Formal contracts manage the interactions of components.

- **Precondition**
  - Assurances that the caller (client) must comply with
  - May not become more strict in sub-classes
- **Postcondition**
  - Assurances that the called (server) must comply with
  - May not become weaker in sub-classes
- **Invariant**
  - Assurances to be complied over all instances of a class
  - Must be invariant also in sub-classes

Conclusion

“objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program”

- Design-By-Contract
- Principle of replaceability!

Stable Extensible Hierarchies

Design Goal	OCP	LSP
Add new feature without touching old code	✓	–
Swap implementations safely	–	✓
Keep unit tests unchanged	✓	✓
Prevent breaking changes	Partial	Strong

Further Reading:

- Covariance and Contravariance in C#: https://www.tutorialsteacher.com/csharp/csharp-covariance-and-contravariance
- dev.to: #3 Liskov Substitution Principle ('L' in SOLID)